

VNN-LIB 2.0: Rigorous Foundations for Neural Network Verification

SONNX Working Group Meeting 2026

Ann Roy¹, Allen Anthony¹, Andrea Gimelli², **Matthew Daggitt¹**

28th of July 2026

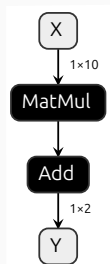
University of Western Australia and University of Genoa

What is VNN-LIB?

VNN-LIB is the standardised format for satisfiability problems over neural networks.

Does there exist an input to a neural network that satisfies a given set of constraints?

```
1 (assert (and (<= 0.0 X0) (<= X0 0.0)))
2 (assert (and (<= 0.0 X1) (<= X1 0.0)))
3 ...
4 (assert (and (<= 0.0 X9) (<= X9 0.0)))
5
6 (assert (<= Y0 Y1))
```



VNN-LIB 1.0 has been used in verifying:

- robustness of vision models,
- safety of cyber-physical controllers.
- correctness of neural certificates.
- etc, etc,

See SAIV 2024-2026 programs

Why was VNN-LIB 2.0 needed?

Problem 1: No formal syntax

Every solver implements their own slightly incompatible parser.

Why was VNN-LIB 2.0 needed?

Problem 1: No formal syntax

Every solver implements their own slightly incompatible parser.

Problem 2: Poor expressivity

Problem must involve *one* network with *one* input and *one* output.

- no hidden layer constraints,
- no multi-modal networks,
- no relational properties.

Why was VNN-LIB 2.0 needed?

Problem 1: No formal syntax

Every solver implements their own slightly incompatible parser.

Problem 2: Poor expressivity

Problem must involve *one* network with *one* input and *one* output.

- no hidden layer constraints,
- no multi-modal networks,
- no relational properties.

Problem 3: No type system

Cannot specify if using reals, or (exotic?) floating-point types.

Why was VNN-LIB 2.0 needed?

Problem 1: No formal syntax

Every solver implements their own slightly incompatible parser.

Problem 2: Poor expressivity

Problem must involve *one* network with *one* input and *one* output.

- no hidden layer constraints,
- no multi-modal networks,
- no relational properties.

Problem 3: No type system

Cannot specify if using reals, or (exotic?) floating-point types.

Problem 4: No semantics

Impossible to formally reason on:

- solver correctness,
- higher-level tool correctness,
- optimisations,
- proof checkers.

Why was VNN-LIB 2.0 needed?

Problem 1: No formal syntax

Every solver implements their own slightly incompatible parser.

Problem 2: Poor expressivity

Problem must involve *one* network with *one* input and *one* output.

- no hidden layer constraints,
- no multi-modal networks,
- no relational properties.

Problem 3: No type system

Cannot specify if using reals, or (exotic?) floating-point types.

Problem 4: No semantics

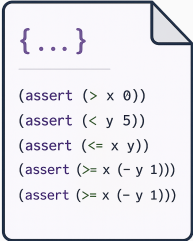
Impossible to formally reason on:

- solver correctness,
- higher-level tool correctness,
- optimisations,
- proof checkers.

VNN-LIB 2.0 addresses all four.

Why is this so hard?

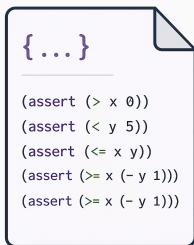
An SMT-LIB query is self-contained:



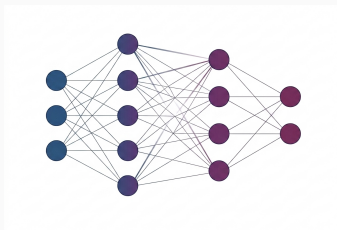
```
{ ... }
```

```
(assert (> x 0))  
(assert (< y 5))  
(assert (<= x y))  
(assert (>= x (- y 1)))  
(assert (>= x (- y 1)))
```

However a VNN-LIB query is not...

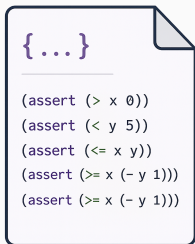


+

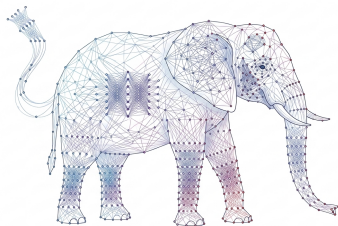


The syntax and semantics of VNNLIB depends on the syntax and semantics of your neural network framework!

However a VNN-LIB query is not...



+



The syntax and semantics of VNNLIB depends on the syntax and semantics of your neural network framework!

The Neural Network Semantics Problem

**No mainstream neural network framework
has a formal semantics!**

The Neural Network Semantics Problem

**No mainstream neural network framework
has a formal semantics!**

and even if we came up with one...

it would get updated every 2 months!

Our solution

Abstraction: A Neural Network Theory

We introduce the notion of an abstract neural network theory, ψ .

This interface provides the minimal set of syntax and semantics necessary for us to define $\text{VNN-LIB}_{2.0}(\psi)$, e.g.

- $\text{VNN-LIB}_{2.0}(\text{ONNX}_{1.22})$
- $\text{VNN-LIB}_{2.0}(\text{ONNX}_{1.23})$
- $\text{VNN-LIB}_{2.0}(\text{PyTorch}_{2.12})$
- etc.

Neural Network Theory: Syntax

The network theory ψ needs to provide the following syntax:

1. `<elementType>` - a set of numeric element types supported by the model format
 - e.g. ONNX element types `float64`, `int32`

Neural Network Theory: Syntax

The network theory ψ needs to provide the following syntax:

1. `<elementType>` - a set of numeric element types supported by the model format
 - e.g. ONNX element types `float64`, `int32`
2. `<tensor>` - the representation of tensors
 - e.g. ONNX `TensorProto` objects.

Neural Network Theory: Syntax

The network theory ψ needs to provide the following syntax:

1. `<elementType>` - a set of numeric element types supported by the model format
 - e.g. ONNX element types `float64`, `int32`
2. `<tensor>` - the representation of tensors
 - e.g. ONNX `TensorProto` objects.
3. `<model>` - the representation of neural network models
 - e.g, the ONNX `ModelProto` objects serialised as `.onnx` files

Neural Network Theory: Syntax

The network theory ψ needs to provide the following syntax:

1. `<elementType>` - a set of numeric element types supported by the model format
 - e.g. ONNX element types `float64`, `int32`
2. `<tensor>` - the representation of tensors
 - e.g. ONNX `TensorProto` objects.
3. `<model>` - the representation of neural network models
 - e.g, the ONNX `ModelProto` objects serialised as `.onnx` files
4. `<nodeOutput>` - a method of referencing node outputs
 - e.g. according to the ONNX standard, any string compliant with the C90 identifier syntax rules.

Neural Network Theory: Semantics

Theory must also provide typing rules, and semantics:

Neural Network Theory: Semantics

Theory must also provide typing rules, and semantics:

1. $\llbracket \cdot \rrbracket_{\text{elementType}}$ - a function that maps each element type to the mathematical set of values it represents,

Neural Network Theory: Semantics

Theory must also provide typing rules, and semantics:

1. $\llbracket \cdot \rrbracket_{\text{elementType}}$ - a function that maps each element type to the mathematical set of values it represents,
2. $\llbracket \cdot \rrbracket_{\text{tensor}}$ - a function mapping a syntactic tensor of type to the mathematical tensor,

Theory must also provide typing rules, and semantics:

1. $\llbracket \cdot \rrbracket_{\text{elementType}}$ - a function that maps each element type to the mathematical set of values it represents,
2. $\llbracket \cdot \rrbracket_{\text{tensor}}$ - a function mapping a syntactic tensor of type to the mathematical tensor,
3. $\llbracket \cdot \rrbracket_{\text{model}}$ - the precise mathematical function represented by the model,

Theory must also provide typing rules, and semantics:

1. $\llbracket \cdot \rrbracket_{\text{elementType}}$ - a function that maps each element type to the mathematical set of values it represents,
2. $\llbracket \cdot \rrbracket_{\text{tensor}}$ - a function mapping a syntactic tensor of type to the mathematical tensor,
3. $\llbracket \cdot \rrbracket_{\text{model}}$ - the precise mathematical function represented by the model,
4. $\llbracket \leq \rrbracket, \dots, \llbracket \times \rrbracket$ - the semantics of basic pointwise comparison and arithmetic operations.

Formal Definition of VNN-LIB 2.0

Formal Definition of VNN-LIB 2.0

$$\begin{aligned}\langle elementType \rangle &\ni \tau ::= \square \\ \langle shape \rangle &\ni s ::= [n_1, \dots, n_k] \quad \text{where } n_i \in \mathbb{N} \\ \langle tensorType \rangle &\ni \delta ::= \langle elementType \rangle \times \langle shape \rangle \\ \langle modelType \rangle &\ni \gamma ::= \langle tensorType \rangle^+ \times \langle tensorType \rangle^+\end{aligned}$$

(a) Abstract grammar for network model types.

$$\begin{aligned}\langle element \rangle &\ni d ::= (-)[0-9]^+ \cdot ([0-9]^+)^+ & \langle tensor \rangle &\ni t ::= \square \\ \langle nodeOutput \rangle &\ni u ::= \square & \langle model \rangle &\ni m ::= \square \\ networkOutputs : \langle model \rangle &\rightarrow \langle nodeOutput \rangle^+ = \square\end{aligned}$$

(b) Abstract grammar and functions for network models.

$$\begin{array}{ccc}\frac{\square}{\vdash d : \tau} \text{ (ELEMENT)} & \frac{\square}{\vdash t : \delta} \text{ (TENSOR)} & \frac{\square}{\vdash m : \gamma} \text{ (MODEL)} \\ \frac{\square}{m \vdash u : \delta} \text{ (NODEOUTPUT)} & \frac{\square}{\vdash m_1 \cong m_2} \text{ (ISOMORPHIC)} & \end{array}$$

Formal Definition of VNN-LIB 2.0

$\langle \text{elementType} \rangle \ni \tau ::= \square$
 $\langle \text{shape} \rangle \ni s ::= [n_1, \dots, n_n] \quad \text{where } n \in \mathbb{N}$
 $\langle \text{tensorType} \rangle \ni \delta ::= \langle \text{elementType} \rangle \times \langle \text{shape} \rangle$
 $\langle \text{modelType} \rangle \ni \gamma ::= \langle \text{tensorType} \rangle^+ \dots \langle \text{tensorType} \rangle^+$

(a) Abstract grammar
 $\langle \text{query} \rangle ::= \langle \text{version} \rangle \langle \text{network} \rangle^+ \langle \text{assert} \rangle^*$
 $\langle \text{version} \rangle ::= (\text{vnnlib-version } [0-9]^+.[0-9]^+)$
 $\langle \text{element} \rangle \ni d ::= (-)[0-9]^+.[0-9]$
 $\langle \text{name} \rangle ::= [A-Za-z][_A-Za-z0-9]^*$
 $\langle \text{nodeOutput} \rangle \ni u ::= \square$
 $\langle \text{network} \rangle ::= (\text{declare-network } \langle \text{name} \rangle \langle \text{equiv} \rangle^? \langle \text{input} \rangle^+ \langle \text{hidden} \rangle^* \langle \text{output} \rangle^+)$
 $\text{networkOutputs} : \langle \text{model} \rangle \rightarrow \langle \text{nodeOutput} \rangle$
 $\langle \text{equiv} \rangle ::= (\text{equal-to } \langle \text{name} \rangle) \mid (\text{isomorphic-to } \langle \text{name} \rangle)$
 $\langle \text{input} \rangle ::= (\text{declare-input } \langle \text{name} \rangle \langle \text{elementType} \rangle^* \langle \text{shape} \rangle)$
 $\langle \text{hidden} \rangle ::= (\text{declare-hidden } \langle \text{name} \rangle \langle \text{elementType} \rangle^* \langle \text{shape} \rangle \langle \text{nodeOutput} \rangle^*)$

(b) Abstract grammar as

$$\frac{\square}{\vdash d : \tau} (\text{ELEMENT}) \quad \frac{\square}{\vdash t : \delta} (\text{TE}) \quad \frac{\square}{\vdash m : \gamma} (\text{MODEL})$$

$$\frac{\square}{m \vdash u : \delta} (\text{NODEOUTPUT}) \quad \frac{\square}{\vdash m_1 \cong m_2} (\text{ISOMORPHIC})$$

Formal Definition of VNN-LIB 2.0

$$\begin{aligned}
 \langle \text{elementType} \rangle &\ni \tau ::= \square \\
 \langle \text{shape} \rangle &\ni s ::= [n, \dots, n] \quad \text{where } n \in \mathbb{N} \\
 \langle \text{tensorType} \rangle &\ni \delta ::= \langle \text{elementType} \rangle \times \langle \text{shape} \rangle \\
 \langle \text{modelType} \rangle &\ni \gamma ::= \langle \text{tensorType} \rangle^+ \dots \langle \text{tensorType} \rangle^+ \\
 &\quad (a) \text{ Abstract grammar} \quad \langle \text{query} \rangle ::= \langle \text{version} \rangle \langle \text{network} \rangle^* \langle \text{assert} \rangle^* \\
 &\quad \langle \text{version} \rangle ::= (\text{vnnlib-version } [0-9]^+.[0-9]^+) \\
 \langle \text{element} \rangle &\ni d \text{ A judgement that a list of network declarations } N \text{ are well-typed in context } \Gamma_1 \\
 &\text{and produces a final context } \Gamma_2 \text{ is written as } \Gamma_1 \vdash N : \Gamma_2 \text{ and is defined recursively} \\
 \langle \text{nodeOutput} \rangle &\ni u \text{ as:} \quad * \langle \text{output} \rangle^+ \\
 \text{networkOutputs} : & \cdot \\
 &\quad (b) \frac{}{\Gamma \vdash [] : \Gamma} \text{NETWORKNIL} \quad \frac{\Gamma_1 \vdash n \quad \Gamma_1[n.\text{name} \rightarrow n] \vdash N : \Gamma_2}{\Gamma_1 \vdash n :: N : \Gamma_2} \text{NETWORKCONS} \\
 \frac{\square}{\vdash d : \tau} (\text{ELEMENT}) & \text{ where } [] \text{ and } :: \text{ represent the standard nil and cons constructors for building up} \\
 \frac{\square}{m \vdash u : \delta} (\text{NODE}) & \text{ a list. In particular, the empty list of the networks is always well-typed under} \\
 & \text{any context and returns the context unaltered. A non-empty list of networks is} \\
 & \text{well-typed if the head of the list can be typed under the current context, and the} \\
 & \text{tail is well-typed under the current context extended by the head of the list.}
 \end{aligned}$$

Formal Definition of VNN-LIB 2.0

$$\begin{aligned}
 \langle \text{elementType} \rangle &\ni \tau ::= \square \\
 \langle \text{shape} \rangle &\ni s ::= [n, \dots, n] \quad \text{where } n \in \mathbb{N} \\
 \langle \text{tensorType} \rangle &\ni \delta ::= \langle \text{elementType} \rangle \times \langle \text{shape} \rangle \\
 \langle \text{modelType} \rangle &\ni \gamma ::= \langle \text{tensorType} \rangle^+ \dots \langle \text{tensorType} \rangle^+ \\
 &\quad (a) \text{ Abstract grammar} \quad \langle \text{query} \rangle ::= \langle \text{version} \rangle \langle \text{network} \rangle^+ \langle \text{assert} \rangle^* \\
 &\quad \langle \text{version} \rangle ::= (\text{vnnlib-version } [0-9]^+.[0-9]^+) \\
 \langle \text{element} \rangle &\ni d \quad \text{A judgement that a list of network declarations } N \text{ are well-typed in context } \Gamma_1 \\
 &\quad \text{and produces a final context } \Gamma_2 \text{ is written as } \Gamma_1 \vdash N : \Gamma_2 \text{ and is defined recursively} \\
 \langle \text{nodeOutput} \rangle &\ni u \quad \text{as:} \quad \text{.* } (\text{output})^+ \\
 \text{net} &\quad \text{Finally, the typing rules for variables ensure that the variable is of the correct} \\
 &\quad \text{type and the indices provided fit within the declared shape of the variable:} \quad \text{NETWORKCONS} \\
 &\quad \frac{\begin{array}{c} \exists v'. \Gamma[v'] = \text{declare-network } v' \in I \ H \ O \\ \exists d \in I \cup H \cup O. \ d.\text{name} = v \wedge d.\text{shape} = s \wedge d.\text{type} = \tau \\ \forall i. l_i < s_i \end{array}}{\Gamma \vdash v [l_1, \dots, l_n] : \tau} \quad \text{VARIABLE} \\
 &\quad \text{for building up} \quad \text{.* } (\text{output})^+ \\
 &\quad \text{ell-typed under} \\
 &\quad \text{: of networks is} \\
 &\quad \text{ontext, and the} \\
 &\quad \text{of the list.}
 \end{aligned}$$

Crucially, therefore variables provide the source of typing for the whole arithmetic expression. One consequence of this is that comparisons that contain no variables (e.g. $(\leq 0.0 \ 1.0)$) have no canonical type and therefore are not typable.

Formal Definition of VNN-LIB 2.0

$$\begin{aligned}
 \langle \text{elementType} \rangle &\ni \tau ::= \square \\
 \langle \text{shape} \rangle &\ni s ::= [n, \dots, n] \quad \text{where } n \in \mathbb{N} \\
 \langle \text{tensorType} \rangle &\ni \delta ::= \langle \text{elementType} \rangle \times \langle \text{shape} \rangle \\
 \langle \text{modelType} \rangle &\ni \gamma ::= \langle \text{tensorType} \rangle^+ \dots \langle \text{tensorType} \rangle^+ \\
 &\quad (a) \text{ Abstract gramm} \quad \langle \text{query} \rangle ::= \langle \text{version} \rangle \langle \text{network} \rangle^* \langle \text{assert} \rangle^* \\
 &\quad \langle \text{version} \rangle ::= (\text{vnnlib-version } [0-9]^+.[0-9]^+) \\
 \langle \text{element} \rangle &\ni d \quad \text{A judgement that a list of network declarations } N \text{ are well-typed in context } \Gamma_1 \\
 &\quad \text{and produces a final context } \Gamma_2 \text{ is written as } \Gamma_1 \vdash N : \Gamma_2 \text{ and is defined recursively} \\
 \langle \text{nodeOutput} \rangle &\ni u \quad \text{as:} \quad * \langle \text{output} \rangle^+ \\
 \text{net} \quad \text{Finally, the typing rules for} \quad \text{Concretely, given a network declaration } n, \text{ and a compatible model } m \text{ and input} \\
 \text{type and the indices provided} \quad \text{assignment } x, \text{ the calculation of the local environment can be defined as follows:} \\
 \frac{\begin{array}{c} \exists v'. \Gamma[v'] : \\ \vdash \delta \end{array}}{\vdash \delta} \quad \Delta(\text{declare-network } v' e I H O, m, X) = \\
 \quad \bigcup_i \Delta_I(I_i) \cup \bigcup_i \Delta_H(H_i) \cup \bigcup_i \Delta_O(O_i, \text{networkOutputs}(m)_i)
 \end{aligned}$$

where

Crucially, therefore variables :
expression. One consequence
(e.g. $\langle \leq 0.0 \ 1.0 \rangle$) have no

$$\begin{aligned}
 \llbracket x \rrbracket &= \{\llbracket X[i.\text{name}] \rrbracket_{\text{tensor}}^\Psi \mid i \in I\} \\
 \Delta_I(\text{declare-input } v \tau s) &= \{v \rightarrow \llbracket X[v] \rrbracket_{\text{tensor}}^\Psi\} \\
 \Delta_H(\text{declare-hidden } v \tau s u) &= \{v \rightarrow \llbracket m \rrbracket_{\text{model}}^\Psi(\llbracket x \rrbracket, u)\} \\
 \Delta_O(\text{declare-output } v \tau s, u) &= \{v \rightarrow \llbracket m \rrbracket_{\text{model}}^\Psi(\llbracket x \rrbracket, u)\}
 \end{aligned}$$

Formal Definition of VNN-LIB 2.0

There is nothing particularly surprising about the semantics of boolean expressions which are interpreted as boolean values:

$$\begin{aligned}
 \llbracket < a_1 a_2 \rrbracket_{\text{bool}}^{\Delta} &= \llbracket < \rrbracket_{\Psi}(\llbracket a_1 \rrbracket_{\text{bool}}^{\Delta}, \llbracket a_2 \rrbracket_{\text{bool}}^{\Delta}) \\
 \llbracket > a_1 a_2 \rrbracket_{\text{bool}}^{\Delta} &= \llbracket > \rrbracket_{\Psi}(\llbracket a_1 \rrbracket_{\text{bool}}^{\Delta}, \llbracket a_2 \rrbracket_{\text{bool}}^{\Delta}) \\
 \llbracket \leq a_1 a_2 \rrbracket_{\text{bool}}^{\Delta} &= \llbracket \leq \rrbracket_{\Psi}(\llbracket a_1 \rrbracket_{\text{bool}}^{\Delta}, \llbracket a_2 \rrbracket_{\text{bool}}^{\Delta}) \\
 \llbracket \geq a_1 a_2 \rrbracket_{\text{bool}}^{\Delta} &= \llbracket \geq \rrbracket_{\Psi}(\llbracket a_1 \rrbracket_{\text{bool}}^{\Delta}, \llbracket a_2 \rrbracket_{\text{bool}}^{\Delta}) \\
 \llbracket = a_1 a_2 \rrbracket_{\text{bool}}^{\Delta} &= \llbracket = \rrbracket_{\Psi}(\llbracket a_1 \rrbracket_{\text{bool}}^{\Delta}, \llbracket a_2 \rrbracket_{\text{bool}}^{\Delta}) \\
 \llbracket ! a_1 a_2 \rrbracket_{\text{bool}}^{\Delta} &= \llbracket \neq \rrbracket_{\Psi}(\llbracket a_1 \rrbracket_{\text{bool}}^{\Delta}, \llbracket a_2 \rrbracket_{\text{bool}}^{\Delta}) \\
 \llbracket \text{and } b_1 \dots b_n \rrbracket_{\text{bool}}^{\Delta} &= \bigwedge_i \llbracket b_i \rrbracket_{\text{bool}}^{\Delta} \\
 \llbracket \text{or } b_1 \dots b_n \rrbracket_{\text{bool}}^{\Delta} &= \bigvee_i \llbracket b_i \rrbracket_{\text{bool}}^{\Delta}
 \end{aligned}$$

□ well-typed if the head of the list can be typed under VARIABLES and the tail is well-typed under the current context extended by the head of the list.

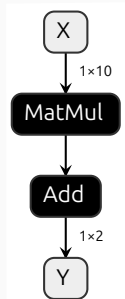
Crucially, therefore variables provide the source of typing for the whole arithmetic expression. One consequence of this is that computations that contain no variables (e.g. $\llbracket \leq 0.0 \ 1.0 \rrbracket$) have no canonical type and therefore hidden (5.3.4):

$$\Delta_0(\text{declare-output } v \ \tau \ s, u) = \{v \mapsto \llbracket m \rrbracket_{\text{model}}^{\Psi}(\llbracket x \rrbracket, u)\}$$

Syntax Examples

Network declarations

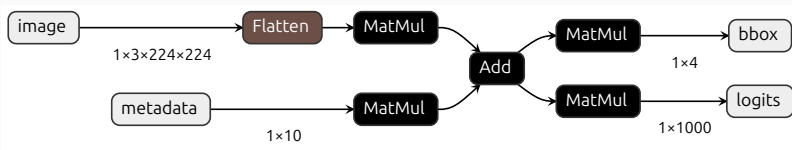
```
1 (vnnlib-version <2.0>)
2
3 (declare-network myNetwork
4   (declare-input  X float32 [1,10])
5   (declare-output Y float32 [1,2])
6 )
7
8 (assert (>= X[0,2] 0.0))
9 (assert (<= X[0,2] 1.0))
10 (assert (<= Y[0,1] 0.5))
```



Multi-modal networks

Multiple input and output declarations allowed:

```
1 (declare-network multi_io_net
2   (declare-input image float32 [1,3,224,224])
3   (declare-input metadata float32 [1,10])
4   (declare-output bbox int16 [1,4])
5   (declare-output logits float32 [1,1000])
6 )
```



Multiple networks

Multiple network declarations can declare hints to the neural network solver:

```
1 (declare-network f
2   (declare-input  A float32 [1,10])
3   (declare-output B float32 [1,2])
4 )
5
6 (declare-network g
7   (isomorphic-to f)
8   (declare-input  C float16 [1,10])
9   (declare-output D float16 [1,2])
10 )
```

Usability and Adoption of the Standard

Real vs Floating point

Commonly known that most neural network solvers use real-arithmetic rather than floating-point...

Our notion of a neural network theory can handle this:

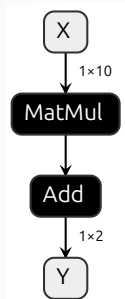
$$\psi + \llbracket \cdot \rrbracket_{\text{realModel}}^\psi \longrightarrow \psi_{\mathbb{R}}$$

We then rederive syntax and semantics over $\mathbb{R}$ for free!

Real-valued VNN-LIB

Using `real` gives the solver permission to use real arithmetic.

```
1 (declare-network myNetwork
2   (declare-input  X real [1,10])
3   (declare-output Y real [1,2])
4 )
5
6 (assert (>= X[0,2] 0.0))
7 (assert (<= X[0,2] 1.0))
8 (assert (<= Y[0,1] 0.5))
```

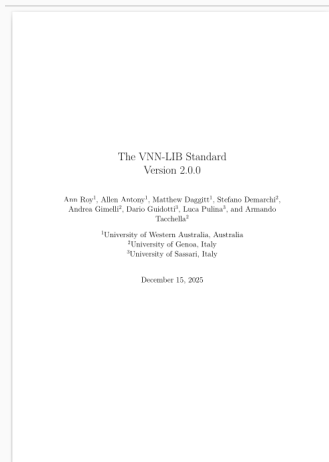


The full standard has been formalised in the Agda theorem prover to ensure consistency.

Other features

Other parts of VNN-LIB 2.0 not covered by the paper:

1. SMT-LIB-style theories and logics.
2. Fast, standardised C++ and Python parser and type-checker.
3. Standardised command-line interface for solvers.



www.vnnlib.org/

Enthusiastic community adoption!

Adoption

Enthusiastic community adoption!

Since its release 6 months ago:

Enthusiastic community adoption!

Since its release 6 months ago:

- 75% of solvers in VNNCOMP 2026 supported VNN-LIB 2.0.

Enthusiastic community adoption!

Since its release 6 months ago:

- 75% of solvers in VNNCOMP 2026 supported VNN-LIB 2.0.
- Community has contributed Julia and Ocaml bindings.

Enthusiastic community adoption!

Since its release 6 months ago:

- 75% of solvers in VNNCOMP 2026 supported VNN-LIB 2.0.
- Community has contributed Julia and Ocaml bindings.
- Rocq bindings apparently incoming in the next few weeks!

Conclusions

We hope this work will:

We hope this work will:

1. Encourage more interesting and creative specifications!

We hope this work will:

1. Encourage more interesting and creative specifications!
2. Drive greater rigour in the field:
 - formal correctness of verifiers
 - unSAT certificates
 - floating point issues etc.

We hope this work will:

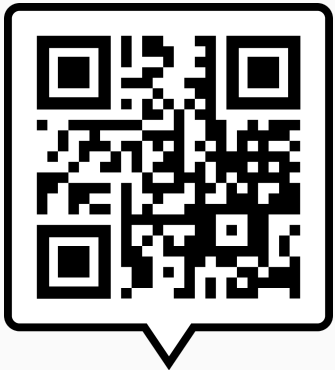
1. Encourage more interesting and creative specifications!
2. Drive greater rigour in the field:
 - formal correctness of verifiers
 - unSAT certificates
 - floating point issues etc.
3. Make the creation of higher-level tools easier!

We hope this work will:

1. Encourage more interesting and creative specifications!
2. Drive greater rigour in the field:
 - formal correctness of verifiers
 - unSAT certificates
 - floating point issues etc.
3. Make the creation of higher-level tools easier!

We are very open to contributions!

Thank you for listening!



Have a look at the standard
now!

With thanks to: Taylor
Johnson, Samuel Teuber, Wen
Kokke, Julien Girard, Guilhem
Ardouin, Augustin Lemesle,
Michele Alberti, Julien
Lehmann, Thomas Flinkow,
Edoardo Manino, Guy Amir,
Omri Isac, Guy Katz, Idan
Refaeli, David Shriver and
Christopher Brix.